

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN

Môn học: HỆ TÍNH TOÁN PHÂN BỐ

**Đề tài: Triển khai Kubernetes Cluster trên
AWS sử dụng kOps**

Giảng viên hướng dẫn : ThS. BÙI THANH BÌNH
Mã lớp học : NT533.P11

Sinh viên thực hiện – Nhóm 2

1. Hồ Hải Dương	MSSV: 21520202
2. Nguyễn Gia Quân	MSSV: 21521327
3. Trần Ngọc Phương Anh	MSSV: 21521839
4. Trương Duy Khôi	MSSV: 22520702

THỦ ĐỨC – 2024

MỤC LỤC

LỜI CẢM ƠN	1
A. LỜI MỞ ĐẦU	2
1. BỐI CẢNH CHỌN ĐỀ TÀI	2
2. MỤC TIÊU ĐỒ ÁN	2
3. PHÂN CHIA CÔNG VIỆC	3
B. NỘI DUNG CHÍNH.....	4
1. ĐIỆN TOÁN ĐÁM MÂY	4
1.1. Tổng quan	4
1.2. Lợi ích của điện toán đám mây	4
1.3. Các mô hình dịch vụ điện toán đám mây	4
1.4. Kiến trúc tổng quát của điện toán đám mây.....	5
2. KUBERNETES.....	6
2.1. Tổng quan	6
2.2. Kiến trúc của Kubernetes	6
2.3. Các thành phần chính trong Kubernetes	7
2.4. Containers trong Kubernetes.....	7
2.5. Ưu điểm khi sử dụng Kubernetes.....	8
3. KOPS	9
3.1. Tổng quan	9
3.2. Các tính năng của kOps.....	9
3.3. Ưu điểm khi sử dụng kOps	10
4. AMAZON WEB SERVICES (AWS).....	10
4.1. Tổng quan	10
4.2. Ưu điểm khi sử dụng AWS	10
4.3. Các dịch vụ AWS được sử dụng trong đồ án.....	11
EC2.....	11
S3	11
IAM.....	11
Route 53	12
Elastic Load Balancer	12
Auto Scaling.....	12
5. HOSTINGER.....	12

5.1. Tổng quan	12
5.2. Ưu điểm khi sử dụng Hostinger	13
6. NGINX.....	14
6.1. Tổng quan	14
6.2. Ưu điểm khi sử dụng Nginx.....	14
7. KIẾN TRÚC HỆ THỐNG	15
7.1. Tổng quan về luồng hoạt động.....	15
7.2. Phân tích chi tiết.....	15
7.2.1. Triển khai Kubernetes cluster bằng kOps	15
7.2.2. Lưu trữ trạng thái cluster trên S3	16
7.2.3. Phân phối lưu lượng qua AWS Elastic Load Balancer (ELB).....	16
7.2.4. Kết nối giữa các thành phần trong Kubernetes cluster.....	16
8. CÁC BƯỚC CÀI ĐẶT HỆ THỐNG.....	17
8.1. Quá trình cài đặt.....	17
8.2. Triển khai Nginx để kiểm tra cluster.....	19
9. CÁC HÌNH ẢNH DEMO QUAN TRỌNG.....	20
9.1. Khởi tạo và cài đặt dịch vụ cần thiết.....	20
9.2. Tạo cluster và triển khai cluster	24
9.3. Kiểm tra hoạt động bằng cách triển khai Nginx Deployment:.....	28
9.4. Kết quả	29
10. TỔNG KẾT ĐỒ ÁN	30
10.1. Kết quả đạt được	30
10.2. Hướng phát triển trong tương lai.....	30
TÀI LIỆU THAM KHẢO.....	31

LỜI CẢM ƠN

Nhóm 2 xin gửi lời cảm ơn chân thành đến Khoa Mạng Máy tính và Truyền thông – Trường Đại học Công nghệ Thông tin vì đã thiết kế và đưa môn Hệ tính toán phân bố vào chương trình giảng dạy. Môn học đã mang đến cho chúng em nền tảng kiến thức vững chắc về các hệ thống phân tán, từ các khái niệm cơ bản đến những kỹ thuật tiên tiến như cluster, grid computing, cloud computing,...

Nhờ đó, chúng em đã hiểu rõ cách thiết kế và triển khai các dịch vụ phân tán như chia sẻ file, liên lạc interprocess, gọi trình từ xa, và kiểm soát đồng thời. Đặc biệt, qua việc thực hiện đồ án "Triển khai cluster Kubernetes trên AWS sử dụng kOps," chúng em đã có cơ hội áp dụng những kiến thức lý thuyết vào thực tế, học được cách triển khai và quản lý cluster Kubernetes với các node control plane và worker trên nền tảng AWS, cũng như tận dụng các tính năng của kOps để đơn giản hóa quá trình cấu hình và quản lý cluster.

Nhóm 2 xin gửi lời cảm ơn đặc biệt tới thầy Bùi Thanh Bình, người đã nhiệt tình giảng dạy và truyền đạt những kiến thức quý báu trong suốt học kỳ. Thầy không chỉ hướng dẫn chúng em nắm bắt các khái niệm lý thuyết mà còn đồng hành và hỗ trợ tận tình trong quá trình thực hiện đồ án, giúp chúng em vượt qua những khó khăn và hoàn thành dự án một cách hiệu quả nhất.

Chúng em hiểu rằng kiến thức là vô tận, trong khi khả năng tiếp thu của mỗi người là giới hạn. Mặc dù đã nỗ lực tìm tòi, nghiên cứu và hoàn thiện đồ án, nhưng chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những góp ý từ thầy để cải thiện và tiến bộ hơn trong tương lai.

Cuối cùng, nhóm 2 xin kính chúc thầy sức khỏe, hạnh phúc và gặt hái nhiều thành công trong sự nghiệp giảng dạy.

Trân trọng,
Nhóm 2.

A. LỜI MỞ ĐẦU

1. BỐI CẢNH CHỌN ĐỀ TÀI

Trong bối cảnh công nghệ thông tin phát triển vượt bậc, điện toán đám mây (cloud computing) đã trở thành một xu hướng tất yếu, mang lại nhiều lợi ích vượt trội trong việc tối ưu hóa tài nguyên, giảm chi phí và cải thiện khả năng mở rộng của các hệ thống. Các doanh nghiệp và tổ chức ngày càng chuyển đổi sang các mô hình đám mây để đáp ứng nhu cầu triển khai dịch vụ nhanh chóng, hiệu quả và đáng tin cậy.

Trong môi trường này, Kubernetes nổi lên như một giải pháp mạnh mẽ cho việc quản lý các ứng dụng container hóa, tự động hóa triển khai và tăng cường khả năng mở rộng. Bên cạnh đó, công cụ kOps đã được sử dụng rộng rãi để đơn giản hóa việc tạo và quản lý cluster Kubernetes trên các nền tảng đám mây, đặc biệt là AWS – một trong những nền tảng điện toán đám mây phổ biến nhất hiện nay.

Đồ án “Triển khai Kubernetes Cluster trên AWS sử dụng kOps” được nhóm 2 thực hiện với mục tiêu áp dụng kiến thức đã học vào thực tế. Việc triển khai không chỉ giúp nhóm hiểu sâu hơn về hệ thống phân tán, các thành phần trong Kubernetes, mà còn cung cấp kinh nghiệm thực tế trong việc sử dụng các công cụ hiện đại để tối ưu hóa hiệu quả quản lý và vận hành hệ thống. Thông qua đồ án, nhóm cũng hy vọng đóng góp những kiến thức hữu ích cho việc phát triển các giải pháp công nghệ trong tương lai.

2. MỤC TIÊU ĐỒ ÁN

BẢNG PHÂN CẤP CÁC MỤC TIÊU	
Mục tiêu	Nội dung chính
Mục tiêu cơ bản	Tìm hiểu các khái niệm cơ bản về Kubernetes và kOps: - Hiểu được Kubernetes là gì, vai trò của nó trong việc quản lý các container, và tại sao Kubernetes là lựa chọn phổ biến cho các hệ thống phân tán. - Tìm hiểu kOps là gì và tại sao nó hỗ trợ tốt cho việc triển khai Kubernetes trên AWS. Chuẩn bị môi trường triển khai: - Tạo tài khoản AWS và cấu hình các dịch vụ cơ bản như EC2, S3, và IAM role. - Cài đặt các công cụ cần thiết như AWS CLI, kubectl, và kOps trên một máy chủ quản trị.
Mục tiêu vừa	Triển khai Kubernetes Cluster (cluster Kubernetes) cơ bản: Sử dụng kOps để tạo một Kubernetes Cluster trên AWS với một node control plane và ít nhất hai node worker.

	- Kiểm tra tính hoạt động của Kubernetes Cluster bằng các lệnh cơ bản như kubectl get nodes. Triển khai ứng dụng thử nghiệm: - Tạo và triển khai một ứng dụng mẫu đơn giản (như Nginx) lên cluster Kubernetes. - Kiểm tra ứng dụng chạy ổn định trên cluster và đảm bảo các container hoạt động đúng yêu cầu.
Mục tiêu nâng cao	Quản lý cluster Kubernetes: - Học cách lưu trữ trạng thái cluster Kubernetes trên S3 và quản lý các cập nhật cluster cơ bản bằng kOps. - Sử dụng các tính năng tự động cơ bản của Kubernetes như self-healing (tự động sửa lỗi khi container gặp sự cố). Đánh giá hiệu quả triển khai: - Đánh giá quá trình triển khai về mặt thời gian, mức độ khó khăn, và kết quả đạt được. - Phân tích những điểm mạnh và hạn chế trong cluster Kubernetes đã triển khai, từ đó rút kinh nghiệm cho các dự án tương lai.

3. PHÂN CHIA CÔNG VIỆC

STT	Tên	MSSV	Công việc	Hoàn thành
1	Hồ Hải Dương	21520202	Thực hiện cài đặt và triển khai, viết báo cáo, làm powerpoint, thuyết trình.	100%
2	Nguyễn Gia Quân	21521327	Thực hiện cài đặt và triển khai, viết báo cáo, làm powerpoint, thuyết trình.	100%
3	Trần Ngọc Phương Anh	21521839	Thực hiện cài đặt và triển khai, viết báo cáo, làm powerpoint, thuyết trình.	100%
4	Trương Duy Khôi	22520702	Thực hiện cài đặt và triển khai, viết báo cáo, làm powerpoint, thuyết trình.	100%

B. NỘI DUNG CHÍNH

1. ĐIỆN TOÁN ĐÁM MÂY

1.1. Tổng quan

Điện toán đám mây (cloud computing) là việc truy cập theo yêu cầu các tài nguyên máy tính – bao gồm máy chủ vật lý hoặc ảo, lưu trữ dữ liệu, khả năng mạng, công cụ phát triển ứng dụng, phần mềm và các công cụ phân tích được hỗ trợ bởi trí tuệ nhân tạo thông qua internet với mô hình thanh toán dựa trên mức sử dụng.

1.2. Lợi ích của điện toán đám mây

- **Hiệu quả về chi phí:** Điện toán đám mây cho phép doanh nghiệp giảm bớt chi phí và công sức liên quan đến việc mua sắm, cài đặt, cấu hình và quản lý cơ sở hạ tầng tại chỗ. Thay vào đó, doanh nghiệp chỉ trả phí cho các tài nguyên và dịch vụ đám mây mà họ sử dụng.
- **Tăng tốc độ và tính linh hoạt:** Với điện toán đám mây, các ứng dụng doanh nghiệp có thể được triển khai trong vài phút, giúp các nhóm phát triển và DevOps tận dụng nhanh chóng các phần mềm và hạ tầng hỗ trợ trên đám mây.
- **Khả năng mở rộng không giới hạn:** Điện toán đám mây cung cấp tính đàn hồi và khả năng tự phục vụ, cho phép doanh nghiệp điều chỉnh tài nguyên theo nhu cầu thực tế, tránh lãng phí và tối ưu hóa hiệu suất.
- **Giá trị chiến lược nâng cao:** Bằng cách sử dụng điện toán đám mây, doanh nghiệp có thể tiếp cận các công nghệ tiên tiến nhất, như trí tuệ nhân tạo và phân tích dữ liệu lớn, để tạo lợi thế cạnh tranh và thúc đẩy đổi mới.

1.3. Các mô hình dịch vụ điện toán đám mây

IaaS (Infrastructure-as-a-Service): Cung cấp quyền truy cập theo yêu cầu vào các tài nguyên cơ bản như máy chủ, lưu trữ và mạng, cho phép doanh nghiệp linh hoạt trong việc quản lý và triển khai ứng dụng.

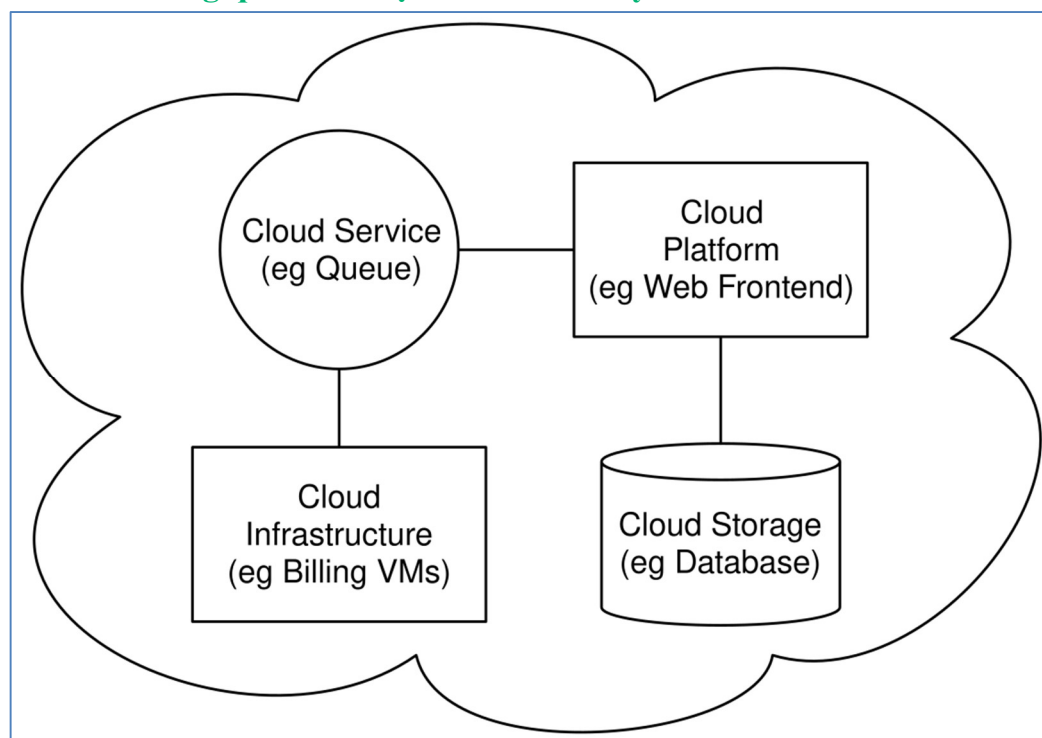
PaaS (Platform-as-a-Service): Cung cấp một nền tảng hoàn chỉnh bao gồm phần cứng, phần mềm và hạ tầng để phát triển, chạy và quản lý ứng dụng, giúp đơn giản hóa quá trình phát triển và triển khai.

SaaS (Software-as-a-Service): Cung cấp các ứng dụng phần mềm sẵn sàng sử dụng thông qua internet, giúp doanh nghiệp tiết kiệm chi phí và thời gian triển khai. SaaS cho phép truy cập dễ dàng mà không cần cài đặt, giúp tiết kiệm chi phí và thời gian triển khai.

BaaS (Backend-as-a-Service): Cung cấp các dịch vụ backend như lưu trữ dữ liệu, quản lý người dùng, thông báo đẩy và tích hợp mạng xã hội thông qua API. BaaS giúp nhà phát triển tập trung vào giao diện người dùng, giảm bớt công việc liên quan đến quản lý backend.

FaaS (Function-as-a-Service): Cho phép phát triển, triển khai và quản lý các hàm ứng dụng nhỏ mà không cần xây dựng và duy trì hạ tầng. FaaS hoạt động dựa trên các sự kiện, tự động mở rộng tài nguyên khi cần thiết, giúp tối ưu hóa chi phí và tăng tính linh hoạt trong phát triển ứng dụng.

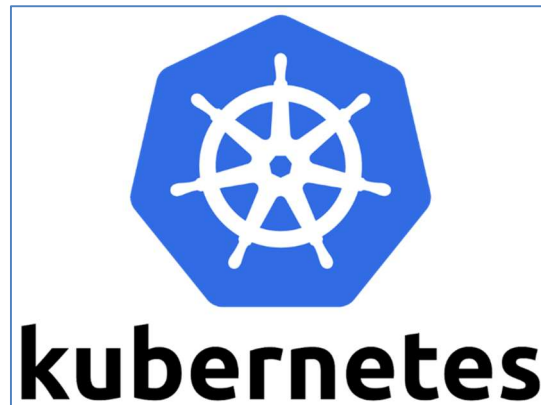
1.4. Kiến trúc tổng quát của điện toán đám mây



Hình 1 – Mô hình kiến trúc tổng quát của điện toán đám mây

Kiến trúc điện toán đám mây bao gồm các thành phần và tiểu thành phần cần thiết cho điện toán đám mây. Các thành phần này thường bao gồm một nền tảng frontend (máy khách dày, máy khách mỏng, di động), các nền tảng backend (máy chủ, lưu trữ), một dịch vụ dựa trên đám mây và một mạng lưới (Internet, Intranet, Intercloud). Kết hợp lại, các thành phần này tạo nên kiến trúc điện toán đám mây, cung cấp dịch vụ linh hoạt và hiệu quả cho người dùng.

2. KUBERNETES



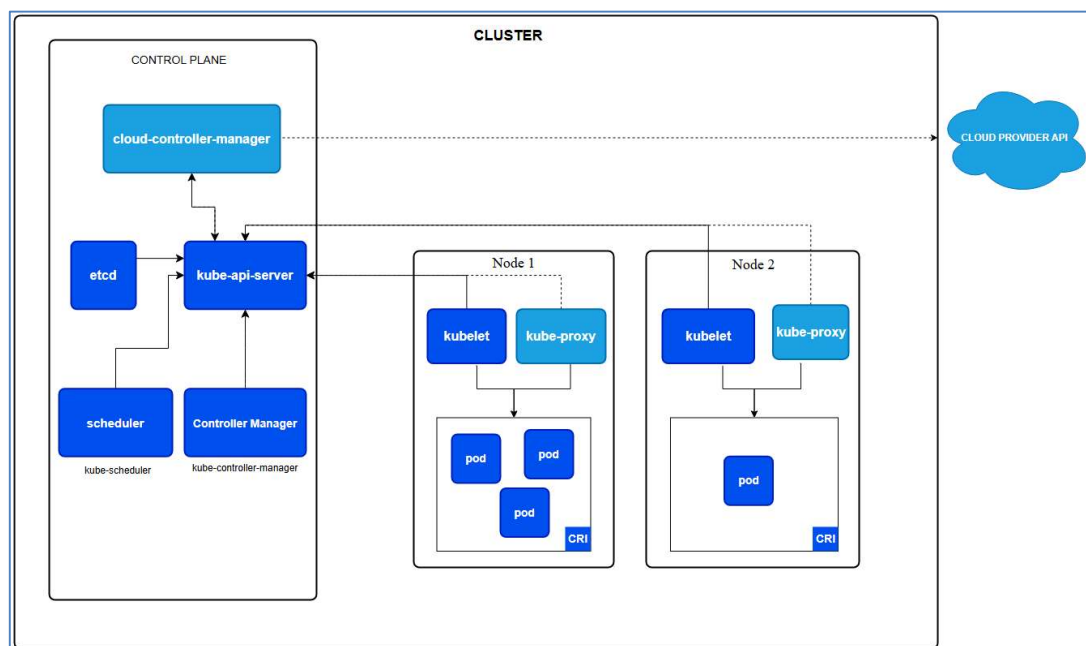
Hình 2 – Logo Kubernetes

2.1. Tổng quan

Kubernetes, hay K8s, là hệ thống mã nguồn mở dùng để tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng dưới dạng container. Kubernetes nhóm các container của một ứng dụng thành các đơn vị logic, giúp dễ quản lý và tìm kiếm. Hệ thống này được xây dựng dựa trên 15 năm kinh nghiệm vận hành khối lượng công việc sản xuất tại Google và các thực tiễn tốt nhất từ cộng đồng.

Kubernetes tạo ra nền tảng để chạy container trên máy chủ vật lý hoặc máy ảo, đồng thời đảm nhận việc phân bổ và quản lý tài nguyên cho container.

2.2. Kiến trúc của Kubernetes



Hình 3 – Kiến trúc của Kubernetes

Kiến trúc của Kubernetes được chia thành hai phần chính:

Control Plane:

- kube-apiserver: Thành phần trung tâm cung cấp API cho toàn bộ cluster, cho phép tương tác và quản lý các tài nguyên.
- etcd: Kho lưu trữ key-value phân tán, đảm bảo tính nhất quán và khả dụng cao cho dữ liệu cấu hình của cluster.
- kube-scheduler: Chịu trách nhiệm phân bổ các pod đến các node phù hợp dựa trên tài nguyên sẵn có và các ràng buộc đã định nghĩa.
- kube-controller-manager: Chạy các controller để giám sát trạng thái của cluster, đảm bảo rằng trạng thái thực tế phù hợp với trạng thái mong muốn.
- cloud-controller-manager: Tích hợp với các nhà cung cấp dịch vụ đám mây, quản lý các tài nguyên như máy ảo, lưu trữ và mạng.

Worker Nodes:

- kubelet: Chạy trên mỗi node, đảm bảo rằng các container trong pod hoạt động đúng cách và theo dõi trạng thái của chúng.
- kube-proxy: Duy trì các quy tắc mạng trên node, cho phép giao tiếp giữa các pod và cân bằng tải cho các dịch vụ.
- Container Runtime: Phần mềm chịu trách nhiệm chạy các container, như Docker hoặc containerd.

2.3. Các thành phần chính trong Kubernetes

- Pod: Đơn vị triển khai nhỏ nhất trong Kubernetes, chứa một hoặc nhiều container chia sẻ cùng một không gian mạng và lưu trữ.
- Service: Định nghĩa một tập hợp các pod và cách truy cập chúng, cung cấp một điểm truy cập ổn định cho các ứng dụng.
- Namespace: Cung cấp không gian tên logic để phân tách và quản lý các tài nguyên trong cluster.
- Deployment: Quản lý việc triển khai và cập nhật các pod, đảm bảo số lượng bản sao mong muốn luôn hoạt động.

2.4. Containers trong Kubernetes

Tổng quan: Containers, hay còn gọi là vùng chứa, là một phương pháp ảo hóa ở cấp độ hệ điều hành, cho phép đóng gói ứng dụng cùng với tất cả các phụ thuộc của nó – bao gồm thư viện, tệp cấu hình và các thành phần cần thiết khác – thành một đơn vị duy nhất. Điều này đảm bảo rằng ứng dụng có thể chạy nhất quán trên nhiều môi trường khác nhau, từ máy tính cá nhân đến hệ thống đám mây.

Trong Kubernetes, containers được triển khai và quản lý thông qua các Pods – đơn vị triển khai nhỏ nhất trong hệ thống. Mỗi Pod có thể chứa một hoặc nhiều containers, chia sẻ cùng một không gian mạng và lưu trữ, cho phép các containers trong cùng một Pod giao tiếp với nhau một cách hiệu quả.

Kubernetes cung cấp các cơ chế để quản lý vòng đời của containers, bao gồm việc khởi động, giám sát, và tự động khởi động lại khi cần thiết. Điều này giúp đảm bảo rằng ứng dụng luôn ở trạng thái sẵn sàng và hoạt động ổn định.

Khi một container chạy trong Kubernetes, nó được cung cấp một số tài nguyên quan trọng như:

- Hệ thống tệp tin: Kết hợp giữa image của container và các volumes được gắn kết.
- Thông tin về container: Bao gồm hostname (thường là tên của Pod chứa container) và các biến môi trường được định nghĩa trong định nghĩa của Pod.
- Thông tin về cluster: Danh sách các services đang chạy khi container được tạo, được cung cấp dưới dạng các biến môi trường hoặc thông qua DNS nếu dịch vụ DNS được kích hoạt.

2.5. Ưu điểm khi sử dụng Kubernetes

- Tự động triển khai và khôi phục: Theo dõi sức khỏe ứng dụng, triển khai từng bước và hoàn nguyên nếu xảy ra sự cố.
- Khám phá dịch vụ và cân bằng tải: Gán IP riêng cho Pods, cung cấp tên DNS duy nhất và hỗ trợ cân bằng tải tự động.
- Điều phối lưu trữ: Tích hợp linh hoạt với nhiều hệ thống lưu trữ như NFS, iSCSI.
- Tự phục hồi: Tự động khởi động lại, thay thế hoặc lên lịch lại container bị lỗi.
- Quản lý bí mật và cấu hình: Cập nhật cấu hình và bí mật mà không cần xây dựng lại container.
- Tự động phân bổ tài nguyên: Điều phối container dựa trên yêu cầu tài nguyên, tối ưu hóa hiệu suất mà không làm giảm tính khả dụng.
- Thực thi lô và CI/CD: Quản lý các khối lượng công việc lô, hỗ trợ triển khai liên tục và thay thế container khi cần.
- Mở rộng linh hoạt: Dễ dàng thay đổi quy mô ứng dụng theo nhu cầu sử dụng.
- Hỗ trợ IPv4/IPv6 dual-stack: Cung cấp khả năng sử dụng cả địa chỉ IPv4 và IPv6.
- Khả năng mở rộng: Thêm tính năng mới mà không ảnh hưởng đến mã nguồn chính.
- Phát triển nhanh chóng: Tạo và triển khai container hiệu quả hơn so với máy ảo.

- Tính nhất quán môi trường: Ứng dụng hoạt động đồng nhất từ máy cá nhân đến môi trường đám mây.
- Đa dạng nền tảng: Hỗ trợ nhiều hệ điều hành và môi trường đám mây.
- Quản lý tập trung vào ứng dụng: Tập trung quản lý và vận hành ứng dụng thay vì hệ điều hành.
- Microservices linh hoạt: Hỗ trợ kiến trúc microservices, giúp chia nhỏ, quản lý và triển khai dễ dàng.
- Cách ly và tối ưu tài nguyên: Đảm bảo hiệu suất ứng dụng ổn định, tận dụng tài nguyên hiệu quả.

3. KOPS



Hình 4 – Logo kOps

3.1. Tổng quan

kOps (Kubernetes Operations) là một công cụ dòng lệnh mã nguồn mở, miễn phí, được thiết kế để cấu hình, duy trì và cung cấp hạ tầng đám mây cần thiết cho việc triển khai các cluster Kubernetes. Với kOps, các nhóm phát triển có thể tự động hóa việc quản lý cluster Kubernetes, bao gồm tạo mới, áp dụng và cập nhật cấu hình cluster một cách hiệu quả.

3.2. Các tính năng của kOps

- Tự động hóa triển khai: kOps cho phép tự động hóa quá trình tạo và cấu hình cluster Kubernetes trên các nền tảng đám mây như AWS, GCP và Azure.
- Quản lý cấu hình: Công cụ này hỗ trợ quản lý cấu hình cluster dưới dạng mã, giúp dễ dàng theo dõi và kiểm soát các thay đổi.
- Cập nhật và nâng cấp: kOps cung cấp khả năng cập nhật và nâng cấp cluster Kubernetes một cách an toàn và hiệu quả, giảm thiểu thời gian gián đoạn dịch vụ.
- Tích hợp với các công cụ DevOps: kOps dễ dàng tích hợp với các công cụ CI/CD, giúp tự động hóa toàn bộ quy trình triển khai và quản lý ứng dụng.

3.3. Ưu điểm khi sử dụng kOps

- Đơn giản hóa quản lý cluster: kOps giúp giảm bớt sự phức tạp trong việc triển khai và quản lý cluster Kubernetes, cho phép tập trung vào phát triển ứng dụng.
- Tăng cường tính nhất quán: Việc quản lý cấu hình dưới dạng mã giúp đảm bảo tính nhất quán giữa các môi trường triển khai.
- Tiết kiệm thời gian và chi phí: Tự động hóa các tác vụ quản lý cluster giúp tiết kiệm thời gian và giảm thiểu chi phí vận hành.

4. AMAZON WEB SERVICES (AWS)



Hình 5 – Logo AWS

4.1. Tổng quan

Amazon Web Services (AWS) là nền tảng điện toán đám mây toàn cầu của Amazon, cung cấp hơn 200 dịch vụ tích hợp từ hạ tầng như điện toán, lưu trữ, cơ sở dữ liệu đến các công nghệ tiên tiến như học máy, trí tuệ nhân tạo (AI), và phân tích dữ liệu lớn. Với mô hình tính phí dựa trên mức sử dụng, AWS phù hợp cho cá nhân, doanh nghiệp, và tổ chức chính phủ triển khai và quản lý ứng dụng hiệu quả.

4.2. Ưu điểm khi sử dụng AWS

Tính năng nổi bật của AWS:

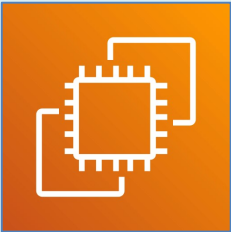
- Tính linh hoạt: AWS cung cấp các dịch vụ như EC2, S3, và Lambda, giúp triển khai ứng dụng trên mọi nền tảng với khả năng tùy chỉnh cao.
- Tính sẵn sàng và bảo mật: Với mạng lưới trung tâm dữ liệu rộng khắp, AWS đảm bảo khả năng chịu lỗi, giảm thiểu rủi ro gián đoạn. Dịch vụ IAM tăng cường bảo mật và quản lý quyền truy cập.
- Khả năng mở rộng: AWS hỗ trợ mở rộng tài nguyên tự động (Auto Scaling) để đáp ứng nhu cầu tăng cao và giảm tài nguyên khi không cần thiết, tối ưu chi phí.

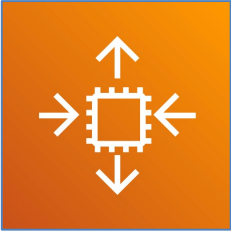
Lợi ích của AWS:

- Tiết kiệm chi phí: Không cần đầu tư vào hạ tầng phần cứng ban đầu.
- Tăng hiệu suất: Tích hợp các công cụ mạnh mẽ giúp triển khai và quản lý ứng dụng nhanh chóng.
- Hỗ trợ đổi mới: AWS cung cấp các công cụ tiên tiến như AI và IoT để thúc đẩy phát triển sản phẩm và giải pháp mới.

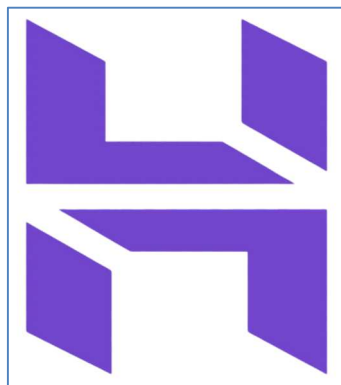
Trong lĩnh vực Hệ thống phân tán hoặc Điện toán đám mây, AWS cung cấp môi trường lý tưởng để thực hành triển khai các ứng dụng container hóa với Kubernetes (EKS), quản lý lưu lượng mạng (Route 53, Elastic Load Balancer), và tối ưu hóa hạ tầng với Auto Scaling. AWS cũng hỗ trợ học tập qua việc xây dựng các bài thực hành về quản lý cơ sở dữ liệu (RDS), phân tích dữ liệu (Redshift), và phát triển ứng dụng serverless (Lambda).

4.3. Các dịch vụ AWS được sử dụng trong đồ án

STT	Dịch vụ	Logo	Tổng quan
1	<i>EC2</i>	 <i>Hình 6</i>	Amazon EC2 cung cấp máy chủ ảo trên đám mây với khả năng mở rộng linh hoạt, cho phép người dùng khởi tạo, cấu hình và quản lý các instance phù hợp với nhu cầu. EC2 hỗ trợ nhiều hệ điều hành và cấu hình tài nguyên (CPU, RAM, lưu trữ), giúp tối ưu hóa hiệu suất ứng dụng. Tính năng Auto Scaling kết hợp giúp tự động điều chỉnh số lượng instance dựa trên lưu lượng truy cập.
2	<i>S3</i>	 <i>Hình 7</i>	Amazon S3 là dịch vụ lưu trữ đối tượng, cho phép lưu trữ và truy xuất dữ liệu ở mọi quy mô với độ bền cao (99.999999999%). S3 hỗ trợ quản lý phiên bản, tích hợp các chính sách quyền truy cập, và tính năng mã hóa dữ liệu nhằm đảm bảo bảo mật. S3 phù hợp cho lưu trữ dữ liệu tĩnh, sao lưu, và phân phối nội dung.
3	<i>IAM</i>	 <i>Hình 8</i>	IAM là dịch vụ quản lý quyền truy cập, giúp kiểm soát chi tiết tài nguyên AWS. Người dùng có thể tạo, quản lý danh tính và gán quyền truy cập theo vai trò hoặc nhóm. IAM hỗ trợ chính sách bảo mật mạnh mẽ, bao gồm xác thực đa yếu tố (MFA), đảm bảo việc sử dụng tài nguyên an toàn và hiệu quả.

4	Route 53	 Hình 9	Route 53 là dịch vụ DNS (Domain Name System) giúp định tuyến lưu lượng đến các ứng dụng trên AWS với độ trễ thấp và tính khả dụng cao. Route 53 hỗ trợ quản lý tên miền, kiểm tra trạng thái ứng dụng và tích hợp cân bằng tải, giúp tối ưu hóa trải nghiệm người dùng.
5	Elastic Load Balancer	 Hình 10	ELB phân phối lưu lượng truy cập đến các instance EC2 hoặc container, đảm bảo tính sẵn sàng và khả năng chịu tải cao. ELB hỗ trợ cân bằng tải ở các lớp HTTP, HTTPS và TCP, đồng thời tích hợp tính năng giám sát sức khỏe để đảm bảo dịch vụ hoạt động ổn định.
6	Auto Scaling	 Hình 11	Auto Scaling tự động điều chỉnh số lượng instance dựa trên nhu cầu thực tế, giúp tối ưu hóa chi phí và hiệu suất. Khi lưu lượng tăng, Auto Scaling mở rộng tài nguyên; khi lưu lượng giảm, tài nguyên sẽ được thu nhỏ, đảm bảo hệ thống luôn hoạt động ở trạng thái tối ưu.

5. HOSTINGER



Hình 12 – Logo Hostinger

5.1. Tổng quan

Hostinger là công ty cung cấp dịch vụ lưu trữ web, được thành lập vào năm 2004 với tên gọi ban đầu là Hosting Media. Trụ sở chính đặt tại Lithuania, công ty hiện có khoảng 900 nhân viên và cung cấp dịch vụ tại hơn 170 quốc gia. Với mạng lưới 10 trung tâm dữ liệu tại 8 quốc gia, Hostinger đảm bảo cung cấp dịch vụ lưu trữ đáng tin cậy và hiệu quả.

5.2. Ưu điểm khi sử dụng Hostinger

Tính năng nổi bật của Hostinger:

- Lưu trữ chia sẻ (Shared Hosting): Giải pháp tiết kiệm, cho phép nhiều trang web sử dụng chung một máy chủ.
- Lưu trữ đám mây (Cloud Hosting): Sử dụng tài nguyên từ nhiều máy chủ, đảm bảo hiệu suất và khả năng chịu lỗi cao.
- Lưu trữ VPS (Virtual Private Server): Dành cho các ứng dụng cần tài nguyên lớn với máy chủ ảo riêng.
- Lưu trữ WordPress: Dịch vụ tối ưu hóa cho các trang web WordPress, bao gồm cập nhật và bảo mật tự động.
- Lưu trữ email (Email Hosting): Dịch vụ email chuyên nghiệp, hỗ trợ tên miền tùy chỉnh.
- Công nghệ LiteSpeed: Sử dụng máy chủ web nhanh và hiệu quả, hỗ trợ tốc độ tải trang vượt trội.
- hPanel: Bảng điều khiển quản lý dịch vụ độc quyền, thân thiện với người dùng.

Lợi ích của Hostinger:

- Chi phí hợp lý: Các gói dịch vụ được thiết kế phù hợp với nhiều đối tượng, từ cá nhân đến doanh nghiệp nhỏ.
- Hiệu suất cao: Công nghệ LiteSpeed và mạng lưới trung tâm dữ liệu toàn cầu đảm bảo tốc độ và độ tin cậy.
- Hỗ trợ toàn cầu: Cung cấp dịch vụ hỗ trợ khách hàng 24/7 qua chat trực tiếp.
- Tính linh hoạt: Đa dạng dịch vụ phù hợp cho cả người mới bắt đầu và chuyên gia.
- Phù hợp cho cả người mới bắt đầu và chuyên gia.

Ứng dụng trong thực tế:

- Lưu trữ và vận hành các trang web cá nhân hoặc doanh nghiệp.
- Xây dựng và quản lý blog hoặc trang WordPress.
- Tích hợp email doanh nghiệp với tên miền tùy chỉnh.
- Tối ưu hóa hiệu suất và bảo mật cho các dự án trực tuyến.

6. NGINX



Hình 13 – Logo Nginx

6.1. Tổng quan

Nginx (phát âm là "engine x") là một máy chủ web mã nguồn mở, được phát triển bởi Igor Sysoev và ra mắt vào năm 2004. Ban đầu, Nginx được thiết kế để giải quyết vấn đề C10k, tức là khả năng xử lý 10.000 kết nối đồng thời, mà các máy chủ web truyền thống gặp khó khăn. Với kiến trúc sự kiện bất đồng bộ, Nginx có thể xử lý một lượng lớn kết nối đồng thời với mức sử dụng tài nguyên thấp.

6.2. Ưu điểm khi sử dụng Nginx

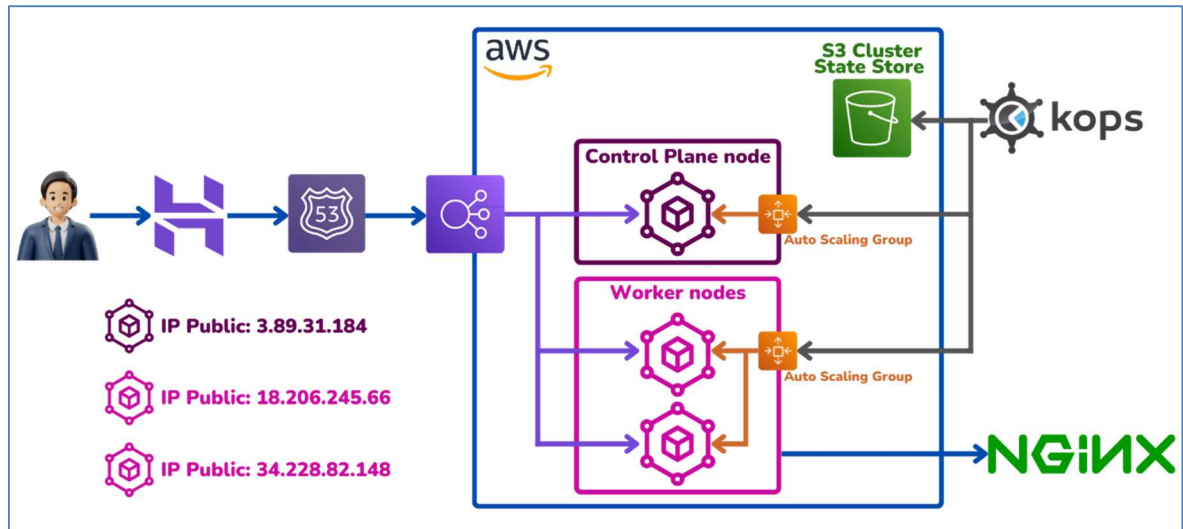
Các tính năng chính của Nginx:

- Máy chủ web: Phục vụ các nội dung tĩnh như HTML, CSS, JavaScript và các tệp phương tiện một cách hiệu quả.
- Reverse proxy và cân bằng tải: Nginx có thể hoạt động như một reverse proxy, chuyển tiếp yêu cầu từ khách hàng đến máy chủ backend, đồng thời thực hiện cân bằng tải để phân phối lượng truy cập đều giữa các máy chủ, tăng hiệu suất và độ tin cậy của hệ thống.
- Bộ nhớ đệm (caching): Hỗ trợ lưu trữ tạm thời các phản hồi từ máy chủ backend, giảm tải và tăng tốc độ phản hồi cho người dùng.
- Hỗ trợ giao thức hiện đại: Nginx hỗ trợ các giao thức như HTTP/2, WebSocket, và gRPC, giúp cải thiện hiệu suất và khả năng mở rộng của ứng dụng web.

Lợi ích của việc sử dụng Nginx:

- Hiệu suất cao: Với kiến trúc bất đồng bộ, Nginx có thể xử lý hàng nghìn kết nối đồng thời mà không gặp vấn đề về hiệu suất.
- Sử dụng tài nguyên hiệu quả: Nginx tiêu thụ ít bộ nhớ và CPU, phù hợp cho cả các hệ thống có tài nguyên hạn chế.
- Dễ cấu hình và mở rộng: Cấu hình của Nginx được quản lý thông qua các tệp văn bản đơn giản, cho phép dễ dàng tùy chỉnh và mở rộng chức năng thông qua các module.

7. KIẾN TRÚC HỆ THỐNG



Hình 14 – Kiến trúc hệ thống của đồ án

7.1. Tổng quan về luồng hoạt động

- **Người dùng** gửi yêu cầu đến ứng dụng thông qua **tên miền** được quản lý trên **Hostinger** và ánh xạ đến **Route 53**.
- **AWS Route 53** xử lý DNS và định tuyến lưu lượng truy cập đến **AWS Elastic Load Balancer (ELB)** qua giao thức **HTTP/HTTPS** trên cổng **80/443**.
- **ELB** tiếp nhận lưu lượng và phân phối đến các **Worker Nodes** trong **Kubernetes cluster**.
- **Nginx Ingress Controller** trong **Kubernetes cluster** nhận lưu lượng từ **ELB** và định tuyến vào đúng **Pods** dựa trên cấu hình **Ingress Rules**.
- Trong quá trình vận hành, **kOps** duy trì và cập nhật cấu hình cụm dựa trên trạng thái lưu trữ trong **AWS S3**. **Worker Nodes** được tự động điều chỉnh số lượng thông qua **AWS Auto Scaling** để đáp ứng nhu cầu tải.
- Toàn bộ hệ thống được bảo mật bởi **AWS IAM**, quản lý quyền truy cập và hạn chế kết nối trái phép.

7.2. Phân tích chi tiết

7.2.1. Triển khai Kubernetes cluster bằng kOps

Sử dụng **kOps** để triển khai và quản lý cụm **Kubernetes (K8s)** trên **AWS**.

- **Thành phần cụm:** Bao gồm 1 **Control Plane Node** (nút điều khiển) và 2 **Worker Nodes** (nút công việc) để chạy các ứng dụng và dịch vụ trong cụm.

- **Giao thức giao tiếp:** Control Plane giao tiếp với các Worker Nodes thông qua **API Server** trên giao thức **HTTPS** qua cổng **6443**. Các thành phần như **kubelet** trên Worker Nodes gửi yêu cầu đến API Server để đảm bảo trạng thái của cụm.
- **Trạng thái cụm:** Trong quá trình triển khai, **kOps** lưu trữ cấu hình và trạng thái của cụm Kubernetes trong một **bucket AWS S3**. Kết nối giữa kOps và S3 sử dụng giao thức **HTTPS** để đảm bảo tính bảo mật và toàn vẹn dữ liệu.

7.2.2. Lưu trữ trạng thái cluster trên S3

- **S3 Cluster State Store:** kOps lưu trữ các tệp cấu hình, trạng thái cluster, và bản sao lưu trên **AWS S3**. Khi thực hiện thao tác **tạo**, **cập nhật** hoặc **nâng cấp** cụm, kOps truy cập vào bucket S3 để lấy hoặc ghi dữ liệu.
- **Bảo mật:** Dữ liệu trên S3 được bảo mật thông qua **AWS IAM**, giới hạn quyền truy cập chỉ cho các thành phần cần thiết. Giao thức truy cập sử dụng **HTTPS** để đảm bảo an toàn khi truyền dữ liệu.

7.2.3. Phân phối lưu lượng qua AWS Elastic Load Balancer (ELB)

Sau khi cụm Kubernetes được triển khai thành công, người dùng có thể tương tác với ứng dụng thông qua **AWS Elastic Load Balancer (ELB)**.

- **Giao thức và cổng:** ELB nhận lưu lượng từ người dùng qua các cổng **80 (HTTP)** hoặc **443 (HTTPS)** và chuyển tiếp đến các Worker Nodes.
- **Cân bằng tải:** ELB phân phối lưu lượng đồng đều giữa các Worker Nodes, đảm bảo hệ thống hoạt động ổn định và tránh tình trạng quá tải cho một node cụ thể.
- **Tích hợp với Nginx:** Trong cụm Kubernetes, **Nginx Ingress Controller** được triển khai để định tuyến lưu lượng từ ELB đến các dịch vụ hoặc Pods bên trong cụm thông qua các **Ingress Rules** dựa trên giao thức **HTTP/HTTPS**.

7.2.4. Kết nối giữa các thành phần trong Kubernetes cluster

Control Plane Node: Quản lý toàn bộ cụm Kubernetes thông qua các thành phần như:

- **kube-apiserver:** Giao diện API trung tâm của cụm, giao tiếp với các thành phần khác qua **HTTPS (cổng 6443)**.
- **etcd:** Lưu trữ trạng thái của cụm Kubernetes.

Worker Nodes: Chạy các ứng dụng dưới dạng **Pods**. Các thành phần chính gồm:

- **kubelet:** Quản lý các container trong Worker Nodes và báo cáo trạng thái về Control Plane qua **HTTPS**.
- **kube-proxy:** Định tuyến lưu lượng mạng đến đúng Pods dựa trên các **iptables** hoặc **IPVS**.

8. CÁC BƯỚC CÀI ĐẶT HỆ THỐNG

8.1. Quá trình cài đặt

- Bước 1: Tạo một máy ảo EC2 chạy hệ điều hành Ubuntu để làm máy chủ quản lý kOps.
- Bước 2: Gán IAM Role cho Instance EC2 với quyền AdministratorAccess để kOps có thể tương tác với các dịch vụ AWS.
- Bước 3: Tạo S3 Bucket để lưu trữ trạng thái và cấu hình của cụm Kubernetes do kOps quản lý.
- Bước 4: Cài đặt AWS CLI

```
sudo apt update

curl      "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip"      -o
"awscliv2.zip"

sudo apt install unzip

unzip awscliv2.zip

sudo ./aws/install

aws --version
```

- Bước 5: Cài đặt Kubectl và kOps

Cài đặt Kubectl:

```
curl      -LO      https://dl.k8s.io/release/$(curl      -L      -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl

chmod +x ./kubectl

sudo mv ./kubectl /usr/local/bin/kubectl

kubectl help
```

Cài đặt kOps:

```
curl -Lo kOps https://github.com/kubernetes/kOps/releases/download/$(curl -s https://api.github.com/repos/kubernetes/kOps/releases/latest | grep tag_name | cut -d '"' -f 4)/kOps-linux-amd64

chmod +x kOps

sudo mv kOps /usr/local/bin/kOps

kOps version
```

- Bước 6: Chuẩn bị tên miền

Kiểm tra tên miền bằng lệnh:

```
nslookup -type=ns <tên-miền>
```

- Bước 7: Tạo cụm Kubernetes bằng kOps

```
kOps create cluster \

--name <tên-miền> \

--state=s3://<tên-s3-bucket> \

--zones=us-east-1a,us-east-1b \

--node-count=2 --node-size=t3.medium \

--master-size=t3.medium --dns-zone <tên-miền> \

--node-volume-size=10 --master-volume-size=10
```

- Bước 8: Tạo cụm Kubernetes

```
kOps update cluster --name <tên-miền> --state=s3://<tên-s3-bucket> --yes --admin
```

- Bước 9: Nâng cấp cụm

```
kOps upgrade cluster --name <tên-miền> --state=s3://<tên-s3-bucket> --yes
```

- Bước 10: Kiểm tra cụm Kubernetes (Quá trình xác thực cluster khoảng 7–10 phút)

```
kOps validate cluster --name <tên-miền> --state=s3://<tên-s3-bucket> --wait 10m
```

- Bước 11: Kiểm tra trạng thái các Node

```
kubectl get nodes
```

- Bước 12: Kết quả Node mẫu, ví dụ:

NODE STATUS

NAME	ROLE	READY
i-00c6cf4a939946c5f	control-plane	True
i-01234c5a23736bd98	node	True
i-0090a21e8b1d3b7bb	node	True

- Bước 13: Xác minh thông tin cụm Kubernetes

```
kubectl cluster-info
```

```
kubectl get nodes
```

8.2. Triển khai Nginx để kiểm tra cluster

- Bước 1: Chạy file cấu hình Deployment mẫu từ Kubernetes

```
kubectl apply -f https://k8s.io/examples/application/deployment.yaml
```

- Bước 2: Hiển thị thông tin Deployment

```
kubectl describe deployment nginx-deployment
```

- Bước 3: Liệt kê các Pod đã tạo

```
kubectl get pods -l app=nginx
```

- Bước 4: Hiển thị chi tiết một Pod cụ thể

```
kubectl describe pod <tên-pod>
```

9. CÁC HÌNH ẢNH DEMO QUAN TRỌNG

9.1. Khởi tạo và cài đặt dịch vụ cần thiết

The screenshot displays the AWS Management Console 'Instances' page. At the top, there's a search bar and a table of instances. The table has columns for Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, and Public IPv4 DNS. Two instances are listed: 'instance-nt533' (Terminated) and 'ec2-nt533' (Running). The details for 'ec2-nt533' are expanded below the table. In the 'Instance summary' section, the 'Public IPv4 address' is highlighted with a red box, showing the value '174.129.146.226' with a link to 'open address'.

Hình 15 – Tạo instance trên AWS

The screenshot shows the 'Select trusted entity' page in the AWS IAM console. The 'Trusted entity type' section has four options: 'AWS service' (selected), 'AWS account', 'SAML 2.0 federation', and 'Custom trust policy'. The 'Use case' section has a text input field containing 'EC2' and a 'Choose a use case for the specified service.' section with 'EC2' selected.

Hình 16

Add permissions Info

Permissions policies (1/1003) Info

Choose one or more policies to attach to your new role.

Filter by Type

All types ▼

Policy name	Type
☒ AdministratorAccess	AWS managed - job function

Hình 17

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

iam-ec2-nt533

Maximum 64 characters. Use alphanumeric and '+', '=', '@', '-', '_' characters.

Hình 18 – Tạo IAM role mới, thêm quyền AdministratorAccess vào role

Instances (1/2) Info

Find Instance by attribute or tag (case-sensitive) All states ▼ Last updated less than a minute ago Connect Instance state ▼ Actions ▲ Launch i

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IP
instance-nt533	i-0f0d1cf06cf8717b1	Terminated	t2.micro	-	View alarms +	us-east-1c	-
☒ ec2-nt533	i-0903bfe61d9777cfd	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1c	ec2-

i-0903bfe61d9777cfd (ec2-nt533)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Change security groups | Get Windows password | Modify IAM role

Connect | View details | Manage instance state | Instance settings | Networking | Security | Image and templates | Monitor and troubleshoot

Hình 19

Modify IAM role Info

Attach an IAM role to your instance.

Instance ID
i-0903bfe61d9777cfd (ec2-nt533)

IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached

iam-ec2-nt533

Hình 20 – Sử dụng IAM role cho Instance đã tạo


```
ubuntu@ip-172-31-91-179: ~  
* Using username "ubuntu".  
* Authenticating with public key "kp-ec2-nt533"  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1015-aws x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
System information as of Sat Nov 30 10:39:45 UTC 2024  
  
System load:  0.01          Processes:            102  
Usage of /:   21.5% of 7.57GB Users logged in:        0  
Memory usage: 20%          IPv4 address for eth0: 172.31.91.179  
Swap usage:   0%  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
New release '24.04.1 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ubuntu@ip-172-31-91-179:~$  
ubuntu@ip-172-31-91-179:~$
```

Hình 21 – SSH vào máy ảo qua địa chỉ IP public

Lưu ý: Khi tạo Instance, cần tạo file secret key, để SSH đến máy ảo, cần thêm file key vào Credential.

```
ubuntu@ip-172-31-91-179:~$ curl "https://awscli.amazonaws.com/awscli-exe-linux-
86_64.zip" -o "awscliv2.zip"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 64.1M  100 64.1M    0     0  103M      0  --:--:-- --:--:-- --:--:-- 103M
ubuntu@ip-172-31-91-179:~$ sudo apt install unzip
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  zip
The following NEW packages will be installed:
  unzip
0 upgraded, 1 newly installed, 0 to remove and 30 not upgraded.
Need to get 175 kB of archives.
After this operation, 386 kB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu jammy-updates/main amd64
zip amd64 6.0-26ubuntu3.2 [175 kB]
Fetched 175 kB in 0s (5145 kB/s)
Selecting previously unselected package unzip.
(Reading database ... 65783 files and directories currently installed.)
Preparing to unpack .../unzip_6.0-26ubuntu3.2_amd64.deb ...
Unpacking unzip (6.0-26ubuntu3.2) ...
Setting up unzip (6.0-26ubuntu3.2) ...
Processing triggers for man-db (2.10.2-1) ...
Scanning processes...
Scanning linux images...
```

Hình 22

```
ubuntu@ip-172-31-91-179:~$ aws --version
aws-cli/2.22.8 Python/3.12.6 Linux/6.8.0-1015-aws exe/x86_64.ubuntu.22
```

Hình 23 – Cài đặt AWS trên máy ảo

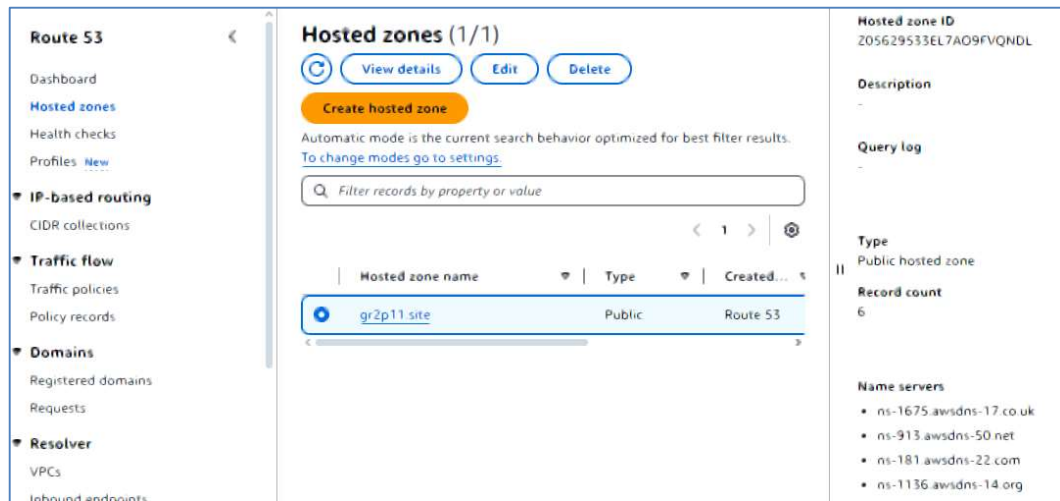
```
curl: try 'curl --help' or 'curl --manual' for more information
-bash: https://dl.k8s.io/release/stable.txt: No such file or directory
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 138  100 138    0     0  1393      0  --:--:-- --:--:-- --:--:-- 1408
100 238  100 238    0     0   934      0  --:--:-- --:--:-- --:--:-- 934
ubuntu@ip-172-31-91-179:~$ curl -LO https://dl.k8s.io/release/$(curl -L -s https
://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 138  100 138    0     0  1389      0  --:--:-- --:--:-- --:--:-- 1380
100 53.7M  100 53.7M    0     0  79.7M      0  --:--:-- --:--:-- --:--:-- 79.7M
ubuntu@ip-172-31-91-179:~$ chmod +x ./kubectl
ubuntu@ip-172-31-91-179:~$ sudo mv ./kubectl /usr/local/bin/kubectl
```

Hình 24 – Cài đặt Kubectl trên máy ảo

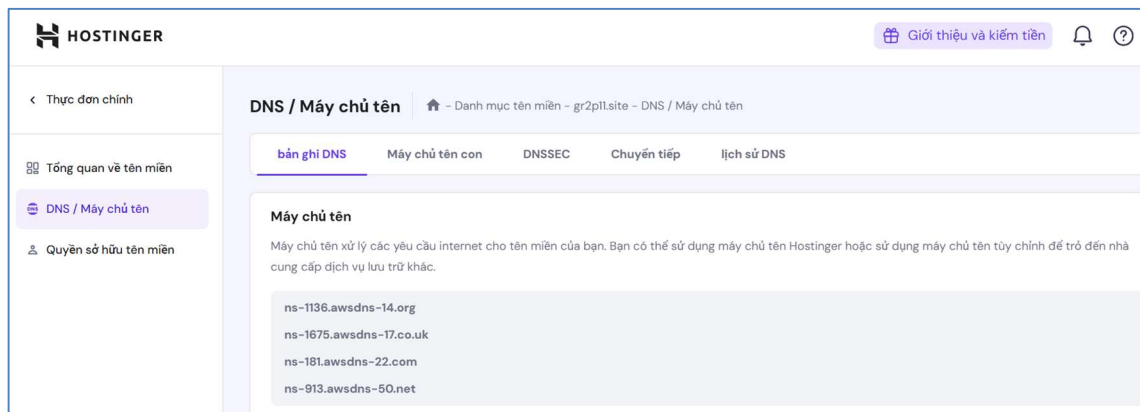
```
ubuntu@ip-172-31-91-179:~$ curl -Lo kops https://github.com/kubernetes/kops/rele
ases/download/$(curl -s https://api.github.com/repos/kubernetes/kops/releases/la
test | grep tag_name | cut -d '"' -f 4)/kops-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0    0      0  --:--:-- --:--:-- --:--:--    0
100 238M  100 238M    0     0  93.6M      0  0:00:02  0:00:02 --:--:-- 111M
ubuntu@ip-172-31-91-179:~$ chmod +x kops
ubuntu@ip-172-31-91-179:~$ sudo mv kops /usr/local/bin/kops
ubuntu@ip-172-31-91-179:~$ kops version
Client version: 1.30.2 (git-v1.30.2)
```

Hình 25 – Cài đặt kOps trên máy ảo

9.2. Tạo cluster và triển khai cluster



Hình 26 – Tạo hostzone trong Route 53

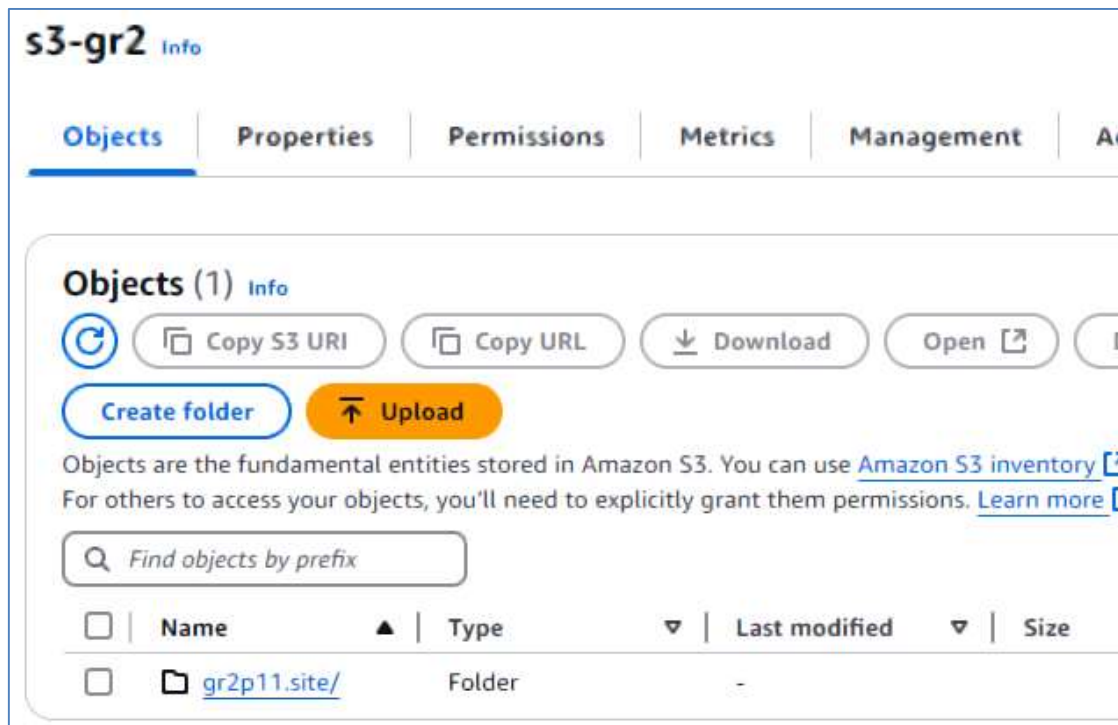


Hình 27 – Để route được các cluster, cần một domain được đổi lại Nameserver trỏ tới Route 53, ở đây là gr2p11.site

```
ubuntu@ip-172-31-6-215:~$ nslookup -type=ns gr2p11.site 8.8.8.8
Server:      8.8.8.8
Address:     8.8.8.8#53

Non-authoritative answer:
gr2p11.site  nameserver = ns-1136.awsdns-14.org.
gr2p11.site  nameserver = ns-1675.awsdns-17.co.uk.
gr2p11.site  nameserver = ns-181.awsdns-22.com.
gr2p11.site  nameserver = ns-913.awsdns-50.net.
```

Hình 28 – Dùng lệnh NSLookup để kiểm tra xem NS record của domain đã trỏ tới NS của AWS chưa



Hình 29 – Tạo bucket trong S3 service

```
ubuntu@ip-172-31-6-215:~$
ubuntu@ip-172-31-6-215:~$
ubuntu@ip-172-31-6-215:~$ kops create cluster \
--name gr2p11.site \
--state=s3://s3-gr2 \
--zones=us-east-1a,us-east-1b \
--node-count=2 --node-size=t3.medium \
--control-plane-size=t3.medium \
--control-plane-volume-size=10 \
--node-volume-size=10 \
--dns-zone gr2p11.site
I1203 12:58:17.062987 2970 new_cluster.go:1454] Cloud Provider ID: "aws"
I1203 12:58:19.021093 2970 subnets.go:224] Assigned CIDR 172.20.0.0/17 to sub
net us-east-1a
I1203 12:58:19.021115 2970 subnets.go:224] Assigned CIDR 172.20.128.0/17 to s
ubnet us-east-1b
Previewing changes that will be made:
I1203 12:58:28.795680 2970 builder.go:312] asset "https://dl.k8s.io/release/v
1.30.6/bin/linux/amd64/kubelet" is not well-known, downloading hash
I1203 12:58:29.024663 2970 builder.go:312] asset "https://dl.k8s.io/release/v
1.30.6/bin/linux/amd64/kubect1" is not well-known, downloading hash
I1203 12:58:29.303713 2970 builder.go:312] asset "https://github.com/contain
```

Hình 30


```

VPCAmazonIPv6CIDRBlock/AmazonIPv6
  VPC          name:gr2p11.site
  Shared       false

VPCDHCPOptionsAssociation/gr2p11.site
  VPC          name:gr2p11.site
  DHCPOptions  name:gr2p11.site

WarmPool/control-plane-us-east-1a.masters.gr2p11.site
  Enabled      false
  MinSize      0
  AutoscalingGroup name:control-plane-us-east-1a.masters.gr2p11.site
  id:control-plane-us-east-1a.masters.gr2p11.site

WarmPool/nodes-us-east-1a.gr2p11.site
  Enabled      false
  MinSize      0
  AutoscalingGroup name:nodes-us-east-1a.gr2p11.site id:nodes-us-east-1a.gr2p11.site

WarmPool/nodes-us-east-1b.gr2p11.site
  Enabled      false
  MinSize      0
  AutoscalingGroup name:nodes-us-east-1b.gr2p11.site id:nodes-us-east-1b.gr2p11.site

Must specify --yes to apply changes

Cluster configuration has been created.

Suggestions:
* list clusters with: kops get cluster
* edit this cluster with: kops edit cluster gr2p11.site
* edit your node instance group: kops edit ig --name=gr2p11.site nodes-us-east-1a
* edit your control-plane instance group: kops edit ig --name=gr2p11.site control-plane-us-east-1a

Finally configure your cluster with: kops update cluster --name gr2p11.site --yes --admin

```

Hình 31 – Tạo cluster với 2 node phụ và 1 node control sử dụng vùng dns gr2p11.site

```

ubuntu@ip-172-31-6-215:~$ kops update cluster --name gr2p11.site --state=s3://s3-gr2 --yes --admin
I1203 13:02:40.016792 3080 builder.go:312] asset "https://dl.k8s.io/release/v1.30.6/bin/linux/amd64/"
I1203 13:02:40.243281 3080 builder.go:312] asset "https://dl.k8s.io/release/v1.30.6/bin/linux/amd64/"
I1203 13:02:40.522497 3080 builder.go:312] asset "https://github.com/containerd/containerd/releases/"
is not well-known, downloading hash
I1203 13:02:42.029774 3080 builder.go:312] asset "https://artifacts.k8s.io/binaries/kops/1.30.2/linux/amd64/"
I1203 13:02:42.536391 3080 builder.go:312] asset "https://dl.k8s.io/release/v1.30.6/bin/linux/arm64/"
I1203 13:02:42.741835 3080 builder.go:312] asset "https://dl.k8s.io/release/v1.30.6/bin/linux/arm64/"
I1203 13:02:43.011710 3080 builder.go:312] asset "https://github.com/containerd/containerd/releases/"
is not well-known, downloading hash
I1203 13:02:43.887478 3080 builder.go:312] asset "https://artifacts.k8s.io/binaries/kops/1.30.2/linux/arm64/"
I1203 13:02:45.732839 3080 builder.go:312] asset "https://artifacts.k8s.io/binaries/kops/1.30.2/linux/arm64/"
I1203 13:02:45.900179 3080 builder.go:312] asset "https://artifacts.k8s.io/binaries/kops/1.30.2/linux/arm64/"
I1203 13:02:46.076096 3080 builder.go:312] asset "https://artifacts.k8s.io/binaries/kops/1.30.2/linux/arm64/"
I1203 13:02:46.260324 3080 builder.go:312] asset "https://artifacts.k8s.io/binaries/kops/1.30.2/linux/arm64/"
I1203 13:03:13.046250 3080 executor.go:113] Tasks: 0 done / 113 total; 45 can run
W1203 13:03:13.198937 3080 vfs_keystorereader.go:143] CA private key was not found
I1203 13:03:13.203937 3080 keypair.go:226] Issuing new certificate: "etcd-peers-ca-events"
I1203 13:03:13.218440 3080 keypair.go:226] Issuing new certificate: "etcd-manager-ca-main"
I1203 13:03:13.235739 3080 keypair.go:226] Issuing new certificate: "apiserver-aggregator-ca"
I1203 13:03:13.255881 3080 keypair.go:226] Issuing new certificate: "etcd-manager-ca-events"

```

Hình 32

```

I1203 13:03:13.296174 3080 keypair.go:226] Issuing new certificate: "etcd-clients-ca"
I1203 13:03:13.459407 3080 keypair.go:226] Issuing new certificate: "service-account"
W1203 13:03:13.477946 3080 vfs_keystorereader.go:143] CA private key was not found
I1203 13:03:13.742108 3080 keypair.go:226] Issuing new certificate: "kubernetes-ca"
I1203 13:03:16.692721 3080 executor.go:113] Tasks: 45 done / 113 total; 25 can run
I1203 13:03:19.350365 3080 executor.go:113] Tasks: 70 done / 113 total; 31 can run
I1203 13:03:21.383464 3080 executor.go:113] Tasks: 101 done / 113 total; 3 can run
I1203 13:03:23.971949 3080 executor.go:113] Tasks: 104 done / 113 total; 6 can run
I1203 13:03:24.849305 3080 executor.go:113] Tasks: 110 done / 113 total; 3 can run
I1203 13:03:25.144271 3080 executor.go:113] Tasks: 113 done / 113 total; 0 can run
I1203 13:03:26.206922 3080 dns.go:235] Pre-creating DNS records
I1203 13:03:26.458241 3080 update_cluster.go:338] Exporting kubeconfig for cluster
kOps has set your kubectl context to gr2p11.site

Cluster is starting. It should be ready in a few minutes.

Suggestions:
* validate cluster: kops validate cluster --wait 10m
* list nodes: kubectl get nodes --show-labels
* ssh to a control-plane node: ssh -i ~/.ssh/id_rsa ubuntu@api.gr2p11.site
* the ubuntu user is specific to Ubuntu. If not using Ubuntu please use the appropriate user
* read about installing addons at: https://kops.sigs.k8s.io/addons.

```

Hình 33 – Cấp quyền cho cluster

```

ubuntu@ip-172-31-6-215:~$ kops upgrade cluster --name gr2p11.site --state=s3://s3-gr2 --yes
No upgrade required

```

Hình 34 – Kiểm tra lại cluster đã cấp quyền đầy đủ chưa

```

ubuntu@ip-172-31-6-215:~$ kops validate cluster --name gr2p11.site --state=s3://s3-gr2 --wait 10m
Validating cluster gr2p11.site

INSTANCE GROUPS
NAME                                ROLE            MACHINETYPE  MIN  MAX  SUBNETS
control-plane-us-east-1a           ControlPlane    t3.medium    1    1    us-east-1a
nodes-us-east-1a                   Node            t3.medium    1    1    us-east-1a
nodes-us-east-1b                   Node            t3.medium    1    1    us-east-1b

NODE STATUS
NAME    ROLE    READY

VALIDATION ERRORS
KIND    NAME      MESSAGE
dns     apiserver Validation Failed

The dns-controller Kubernetes deployment has not updated the Kubernetes cluster's API DNS entry to the correct
controller address that kops creates: 203.0.113.123. Please wait about 5-10 minutes for a control plane node to
propagate. The protokube container and dns-controller deployment logs may contain more diagnostic information.
or a kops Kubernetes cluster to start.

```

Hình 35 – Validate cluster, đợi một lúc lâu (7–10 phút)

```

INSTANCE GROUPS
NAME                                ROLE                MACHINETYPE    MIN    MAX    SUBNETS
control-plane-us-east-1a           ControlPlane        t3.medium      1      1      us-east-1a
nodes-us-east-1a                   Node                t3.medium      1      1      us-east-1a
nodes-us-east-1b                   Node                t3.medium      1      1      us-east-1b

NODE STATUS
NAME                                ROLE                READY
i-0090a21e8b1d3b7bb               node                True
i-00c6cf4a939946c5f               control-plane      True
i-01234c5a23736bd98               node                True

Your cluster gr2p11.site is ready
ubuntu@ip-172-31-6-215:~$
ubuntu@ip-172-31-6-215:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
i-0090a21e8b1d3b7bb               Ready    node     70s    v1.30.6
i-00c6cf4a939946c5f               Ready    control-plane 3m46s  v1.30.6
i-01234c5a23736bd98               Ready    node     75s    v1.30.6

```

Hình 36 – Sau khi validate cluster, tạo thành công 3 node

9.3. Kiểm tra hoạt động bằng cách triển khai Nginx Deployment:

```

ubuntu@ip-172-31-6-215:~$ kubectl apply -f https://k8s.io/examples/application/deployment.yaml
deployment.apps/nginx-deployment created
ubuntu@ip-172-31-6-215:~$ kubectl describe deployment nginx-deployment
Name:                               nginx-deployment
Namespace:                           default
CreationTimestamp:                   Tue, 03 Dec 2024 13:21:39 +0000
Labels:                               <none>
Annotations:                         deployment.kubernetes.io/revision: 1
Selector:                             app=nginx
Replicas:                             2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:                         RollingUpdate
MinReadySeconds:                     0
RollingUpdateStrategy:               25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
    nginx:
      Image:          nginx:1.14.2
      Port:            80/TCP
      Host Port:       0/TCP
      Environment:    <none>
      Mounts:          <none>
      Volumes:         <none>
      Node-Selectors:  <none>
      Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: <none>
NewReplicaSet:  nginx-deployment-77d8468669 (2/2 replicas created)
Events:
  Type           Reason              Age    From                      Message
  ----           -
  Normal         ScalingReplicaSet   17s    deployment-controller     Scaled up replica set nginx-deployment-77d8468669 to 2

```

Hình 37 – Dùng Nginx Deployment vào cluster, xem thông tin deployment

```

ubuntu@ip-172-31-6-215:~$ kubectl get pods -l app=nginx
NAME                                READY    STATUS    RESTARTS   AGE
nginx-deployment-77d8468669-csg4w   1/1      Running   0           67s
nginx-deployment-77d8468669-j65pj   1/1      Running   0           67s

```

Hình 38 – Xem danh sách Pods

```

ubuntu@ip-172-31-6-215:~$ kubectl describe pod nginx-deployment-77d8468669-csg4w
Name:          nginx-deployment-77d8468669-csg4w
Namespace:     default
Priority:       0
Service Account: default
Node:          i-01234c5a23736bd98/172.20.177.153
Start Time:    Tue, 03 Dec 2024 13:21:39 +0000
Labels:        app=nginx
               pod-template-hash=77d8468669
Annotations:   kubernetes.io/limit-ranger: LimitRanger plugin set: cpu request for container nginx
Status:        Running
IP:            100.96.1.1
IPs:
  IP:          100.96.1.1
Controlled By: ReplicaSet/nginx-deployment-77d8468669
Containers:

```

Hình 39

```

Containers:
  nginx:
    Container ID:   containerd://a82e7d538clba3d0b65549abc5ab2f35ec9604c6c7c126fa1833db4e0d0c221b
    Image:          nginx:1.14.2
    Image ID:       docker.io/library/nginx@sha256:f7988fb6c02e0ce69257d9bd9cf37ae20a60f1df7563c3a2a6abe24160306b8d
    Port:          80/TCP
    Host Port:     0/TCP
    State:         Running
      Started:     Tue, 03 Dec 2024 13:21:43 +0000
    Ready:         True
    Restart Count:  0
    Requests:
      cpu:         100m
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-w92w8 (ro)
Conditions:
  Type                     Status
  PodReadyToStartContainers True
  Initialized              True
  Ready                    True
  ContainersReady          True
  PodScheduled             True
Volumes:
  kube-api-access-w92w8:
    Type:              Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:       kube-root-ca.crt
    ConfigMapOptional:   <nil>
    DownwardAPI:        true
QoS Class:              Burstable
Node-Selectors:          <none>
Tolerations:             node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                        node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason      Age   From          Message
  ----     -
  Normal   Scheduled   3m54s default-scheduler Successfully assigned default/nginx-deployment-77d8468669-csg4w to i-01
  Normal   Pulled      3m53s kubelet       Pulling image "nginx:1.14.2"
  Normal   Pulled      3m50s kubelet       Successfully pulled image "nginx:1.14.2" in 2.853s (2.853s including wa
  Normal   Created     3m50s kubelet       Created container nginx
  Normal   Started     3m50s kubelet       Started container nginx

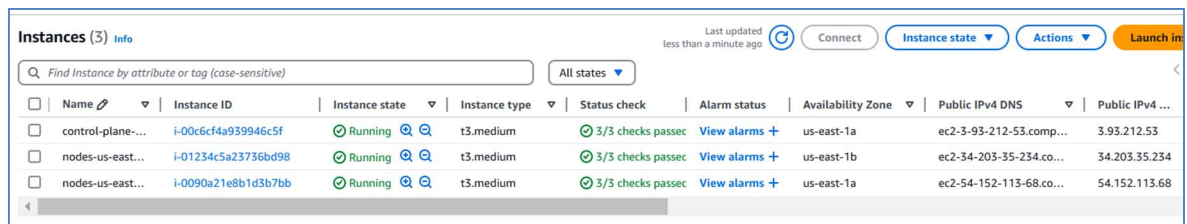
```

Hình 40 – Xem thông tin Pods

9.4. Kết quả

Instances (1) Info										
Find Instance by attribute or tag (case-sensitive)				All states	Last updated less than a minute ago					
					Connect	Instance state	Actions	Launch instances		
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☐	ec2-gr2p11	i-0fca3a6512a0a1676	Running	t2.micro	2/2 checks passed	View alarms	ap-southeast-2a	ec2-13-210-222-12.ap-...	13.210.222.12	-

Hình 41



	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐	control-plane-...	i-00c6c4a939946c5f	Running	t3.medium	3/3 checks passed	View alarms +	us-east-1a	ec2-3-93-212-53.comp...	3.93.212.53
☐	nodes-us-east-...	i-01234c5a23736bd98	Running	t3.medium	3/3 checks passed	View alarms +	us-east-1b	ec2-34-203-35-234.co...	34.203.35.234
☐	nodes-us-east-...	i-0090a21e8b1d3b7bb	Running	t3.medium	3/3 checks passed	View alarms +	us-east-1a	ec2-54-152-113-68.co...	54.152.113.68

Hình 42 – Đã tạo ra được 3 node, kiểm tra node đã xuất hiện chưa

10. TỔNG KẾT ĐỒ ÁN

10.1. Kết quả đạt được

Triển khai cụm Kubernetes trên AWS:

- Tạo thành công 1 Control Plane Node và 2 Worker Nodes thông qua kOps.
- Lưu trữ trạng thái và cấu hình cụm trong AWS S3 Bucket.
- Giao tiếp giữa các thành phần trong cụm thông qua kube-apiserver và các dịch vụ Kubernetes.

Tích hợp các dịch vụ hỗ trợ:

- Sử dụng AWS Route 53 để định tuyến tên miền.
- Triển khai Elastic Load Balancer (ELB) để cân bằng tải lưu lượng truy cập đến các Worker Nodes.
- Cài đặt và sử dụng Nginx Ingress Controller để định tuyến và quản lý lưu lượng vào cụm Kubernetes.

Kiểm tra hệ thống:

- Triển khai ứng dụng mẫu như Nginx để xác minh tính ổn định và hoạt động của cụm.
- Kiểm tra trạng thái các nodes và pods, đảm bảo tất cả đều hoạt động đúng như mong đợi.

10.2. Hướng phát triển trong tương lai

- Tối ưu cụm Kubernetes: Nâng cấp phiên bản và tối ưu tài nguyên.
- Triển khai ứng dụng microservices: Tận dụng khả năng mở rộng linh hoạt.
- Tích hợp CI/CD: Tự động hóa triển khai bằng Jenkins hoặc GitHub Actions.
- Tăng cường bảo mật: Sử dụng Network Policies và RBAC.
- Giám sát hệ thống: Tích hợp Prometheus và Grafana để theo dõi hiệu suất.

TÀI LIỆU THAM KHẢO

1. IBM. (n.d.). *Cloud computing*. IBM. Retrieved December 5, 2024, from <https://www.ibm.com/topics/cloud-computing>
2. Kubernetes Documentation. (n.d.). *Overview*. Kubernetes.io. Retrieved December 5, 2024, from <https://kubernetes.io/docs/concepts/overview/>
3. GeeksforGeeks. (n.d.). *Kubernetes kOps*. Retrieved December 5, 2024, from <https://www.geeksforgeeks.org/kubernetes-kops/>
4. Wikipedia contributors. (n.d.). *Amazon Web Services*. Wikipedia. Retrieved December 5, 2024, from https://en.wikipedia.org/wiki/Amazon_Web_Services
5. Wikipedia contributors. (n.d.). *Hostinger*. Wikipedia. Retrieved December 5, 2024, from <https://en.wikipedia.org/wiki/Hostinger>
6. GeeksforGeeks. (n.d.). *What is Nginx Web Server and how to install it?* Retrieved December 5, 2024, from <https://www.geeksforgeeks.org/what-is-nginx-web-server-and-how-to-install-it/>
7. Simplilearn. (n.d.). *AWS Tutorial for Beginners | AWS Full Course*. YouTube. Retrieved December 5, 2024, from <https://www.youtube.com/watch?v=SWShhJ4138I>